

INCREMENTAL IMPLEMENTATION - NUMERICAL SOLUTION

Complete Step-by-Step Walkthrough

PROBLEM SCENARIO

Setup

- **5-armed bandit** with 20 total plays
 - **True reward distributions** (unknown to agent):
 - Arm 0: Probability 0.30 → Average reward 3.0
 - Arm 1: Probability 0.50 → Average reward 5.0
 - Arm 2: Probability 0.60 → Average reward 6.0 **OPTIMAL**
 - Arm 3: Probability 0.20 → Average reward 2.0
 - Arm 4: Probability 0.45 → Average reward 4.5
 - **Strategy:** -Greedy ($\epsilon = 0.2$) with incremental Q updates
 - **Key Difference:** Use incremental formula instead of storing all rewards
-

WHY INCREMENTAL UPDATE MATTERS

Traditional Approach (Store All Rewards)

Rewards from Arm 2: [10, 0, 10, 10, 0, 10]
 $Q(2) = (10 + 0 + 10 + 10 + 0 + 10) / 6 = 40/6 = 6.67$

Memory required per arm: Store all 6 rewards

Incremental Approach (Update One at a Time)

$$Q_{\text{new}} = Q_{\text{old}} + (\text{reward} - Q_{\text{old}}) / N_{\text{new}}$$

Start: $Q = 0, N = 0$

After reward 10: $Q = 0 + (10 - 0) / 1 = 10, N = 1$

After reward 0: $Q = 10 + (0 - 10) / 2 = 5, N = 2$

After reward 10: $Q = 5 + (10 - 5) / 3 = 6.67, N = 3$

...

Memory required: Only Q and N per arm (no history!)

Both give same answer, but incremental uses $O(1)$ memory instead of $O(T)$!

ALGORITHM DESCRIPTION

Incremental Q-Value Update Formula

$$Q_{new}(a) = Q_{old}(a) + \frac{R_t - Q_{old}(a)}{N_{new}(a)}$$

Equivalently:

$$Q_{new}(a) = \frac{Q_{old}(a) \times N_{old}(a) + R_t}{N_{new}(a)}$$

Understanding the Incremental Formula

Breaking it down:

$$Q_{new} = Q_{old} + \text{error} / N_{new}$$

error = $R_t - Q_{old}$ (surprise: how much better/worse than expected)
N_new = N_old + 1 (count increases by 1)

Step size = $1 / N_{new}$ (decreases over time)

Intuition: - First reward (N=1): step = 1.0 → Q changes a lot - After 10 tries (N=10): step = 0.1 → Q changes less - After 100 tries (N=100): step = 0.01 → Q barely changes - Natural stabilization!

COMPLETE NUMERICAL WALKTHROUGH

Using same rewards as -Greedy but showing incremental calculations:

INITIAL STATE (Play 0)

Arm	Q(a)	N(a)	Error Formula
0	0.0	0	N/A
1	0.0	0	N/A
2	0.0	0	N/A
3	0.0	0	N/A
4	0.0	0	N/A

DETAILED PLAYS 1-20 WITH INCREMENTAL UPDATES

PLAY 1

Selection: Arm 0 (via -greedy) **Reward:** 0

Incremental Update for Q(0):

$N_{\text{old}}(0) = 0$
 $N_{\text{new}}(0) = 0 + 1 = 1$

$Q_{\text{old}}(0) = 0.0$
 $\text{error} = 0 - 0.0 = 0$
 $Q_{\text{new}}(0) = 0.0 + 0 / 1 = 0.0$

State: $Q = [0.0, 0.0, 0.0, 0.0, 0.0]$, $N = [1, 0, 0, 0, 0]$

PLAY 2

Selection: Arm 3 (via random explore) **Reward:** 0

Incremental Update for Q(3):

$N_{\text{old}}(3) = 0$
 $N_{\text{new}}(3) = 0 + 1 = 1$

$Q_{\text{old}}(3) = 0.0$
 $\text{error} = 0 - 0.0 = 0$
 $Q_{\text{new}}(3) = 0.0 + 0 / 1 = 0.0$

State: $Q = [0.0, 0.0, 0.0, 0.0, 0.0]$, $N = [1, 0, 0, 1, 0]$

PLAY 3

Selection: Arm 1 (via exploit) **Reward:** 10

Incremental Update for Q(1):

$N_{\text{old}}(1) = 0$
 $N_{\text{new}}(1) = 0 + 1 = 1$

$Q_{\text{old}}(1) = 0.0$
 $\text{error} = 10 - 0.0 = 10$ (Positive surprise! Better than expected)
 $\text{step_size} = 1 / 1 = 1.0$ (First try: big step)
 $Q_{\text{new}}(1) = 0.0 + 10 \times 1.0 = 10.0$

State: $Q = [0.0, 10.0, 0.0, 0.0, 0.0]$, $N = [1, 1, 0, 1, 0]$ **Total Reward:** 10

PLAY 4

Selection: Arm 1 (via exploit, highest Q) **Reward:** 0

Incremental Update for Q(1):

$N_{old}(1) = 1$
 $N_{new}(1) = 1 + 1 = 2$

 $Q_{old}(1) = 10.0$
 $error = 0 - 10.0 = -10$ (Negative surprise! Worse than expected)
 $step_size = 1 / 2 = 0.5$ (Second try: smaller step)
 $Q_{new}(1) = 10.0 + (-10) \times 0.5 = 10.0 - 5.0 = 5.0$
State: $Q = [0.0, 5.0, 0.0, 0.0, 0.0]$, $N = [1, 2, 0, 1, 0]$ **Total Reward:** 10

PLAY 5

Selection: Arm 2 (via random explore) **Reward:** 10

Incremental Update for Q(2):

$N_{old}(2) = 0$
 $N_{new}(2) = 0 + 1 = 1$

$Q_{old}(2) = 0.0$
 $error = 10 - 0.0 = 10$
 $step_size = 1 / 1 = 1.0$ (First try)
 $Q_{new}(2) = 0.0 + 10 \times 1.0 = 10.0$

State: $Q = [0.0, 5.0, 10.0, 0.0, 0.0]$, $N = [1, 2, 1, 1, 0]$ **Total Reward:** 20
CRITICAL: Found optimal arm!

PLAY 6

Selection: Arm 2 (via exploit, highest $Q=10.0$) **Reward:** 10

Incremental Update for Q(2):

$N_{old}(2) = 1$
 $N_{new}(2) = 1 + 1 = 2$

$Q_{old}(2) = 10.0$
 $error = 10 - 10.0 = 0$ (Matched expectations)
 $step_size = 1 / 2 = 0.5$ (Second try)
 $Q_{new}(2) = 10.0 + 0 \times 0.5 = 10.0$

State: $Q = [0.0, 5.0, 10.0, 0.0, 0.0]$, $N = [1, 2, 2, 1, 0]$ **Total Reward:** 30

PLAY 7

Selection: Arm 4 (via random explore) **Reward:** 0

Incremental Update for Q(4):

$$N_{\text{old}}(4) = 0$$

$$N_{\text{new}}(4) = 0 + 1 = 1$$

$$Q_{\text{old}}(4) = 0.0$$

$$\text{error} = 0 - 0.0 = 0$$

$$\text{step_size} = 1 / 1 = 1.0$$

$$Q_{\text{new}}(4) = 0.0 + 0 \times 1.0 = 0.0$$

State: $Q = [0.0, 5.0, 10.0, 0.0, 0.0]$, $N = [1, 2, 2, 1, 1]$ **Total Reward:** 30

PLAY 8

Selection: Arm 2 (via exploit) **Reward:** 10

Incremental Update for Q(2):

$$N_{\text{old}}(2) = 2$$

$$N_{\text{new}}(2) = 2 + 1 = 3$$

$$Q_{\text{old}}(2) = 10.0$$

$$\text{error} = 10 - 10.0 = 0$$

$$\text{step_size} = 1 / 3 = 0.33 \text{ (Third try: even smaller step)}$$

$$Q_{\text{new}}(2) = 10.0 + 0 \times 0.33 = 10.0$$

State: $Q = [0.0, 5.0, 10.0, 0.0, 0.0]$, $N = [1, 2, 3, 1, 1]$ **Total Reward:** 40

PLAY 9

Selection: Arm 2 (via exploit) **Reward:** 0

Incremental Update for Q(2):

$$N_{\text{old}}(2) = 3$$

$$N_{\text{new}}(2) = 3 + 1 = 4$$

$$Q_{\text{old}}(2) = 10.0$$

$$\text{error} = 0 - 10.0 = -10 \text{ (Disappointment)}$$

$$\text{step_size} = 1 / 4 = 0.25 \text{ (Fourth try)}$$

$$Q_{\text{new}}(2) = 10.0 + (-10) \times 0.25 = 10.0 - 2.5 = 7.5$$

State: $Q = [0.0, 5.0, 7.5, 0.0, 0.0]$, $N = [1, 2, 4, 1, 1]$ **Total Reward:** 40

PLAY 10

Selection: Arm 2 (via exploit) **Reward:** 10

Incremental Update for Q(2):

$N_{\text{old}}(2) = 4$

$N_{\text{new}}(2) = 4 + 1 = 5$

$Q_{\text{old}}(2) = 7.5$

$\text{error} = 10 - 7.5 = 2.5$ (Positive surprise again)

$\text{step_size} = 1 / 5 = 0.2$ (Fifth try)

$Q_{\text{new}}(2) = 7.5 + 2.5 \times 0.2 = 7.5 + 0.5 = 8.0$

State: $Q = [0.0, 5.0, 8.0, 0.0, 0.0]$, $N = [1, 2, 5, 1, 1]$ **Total Reward:** 50

PLAYS 11-20 WITH STEP SIZE DECAY VISIBLE

Key Insight: Step Size Naturally Decreases

Step size = $1/N$, so:

Play	Arm	Reward	N Before	Step Size (1/N)	Error	Change	Q New
11	0	0	1	1/1 = 1.0	-0	0	0.0
12	2	10	6	1/6 0.17	2.0	+0.33	8.33
13	2	10	7	1/7 0.14	1.67	+0.24	8.57
14	1	10	3	1/3 0.33	5.0	+1.67	6.67
15	2	0	8	1/8 = 0.125	-8.57	-1.07	7.50
16	2	10	9	1/9 0.11	2.5	+0.28	7.78
17	4	10	2	1/2 = 0.5	10.0	+5.0	5.0
18	2	10	10	1/10 = 0.1	2.22	+0.22	8.0
19	2	0	11	1/11 0.09	-8.0	-0.73	7.27
20	2	10	12	1/12 0.08	2.73	+0.23	7.50

Observation: As N increases, step size ($1/N$) decreases $\rightarrow$ changes get smaller

FINAL STATE (After Play 20)

Q-Value Estimates with Incremental Updates

Arm	Final Q	N(a)	Arms Tried	Step Size at End (1/N)
0	0.0	1	1x	1.0

Arm	Final Q	N(a)	Arms Tried	Step Size at End (1/N)
1	6.67	3	3x	0.33
2	7.50	12	12x	0.083
3	0.0	1	1x	1.0
4	5.0	2	2x	0.5

Memory Usage Comparison

Traditional (store all rewards):

Total rewards stored: $1 + 3 + 12 + 1 + 2 = 19$ values

Plus Q estimates: 5 values

Total: 24 values

Incremental (only Q and N):

Q values: 5

N values: 5

Total: 10 values

Memory saved: $24 \rightarrow 10$ (58% reduction!)

In large systems: Could be 50x or 100x reduction!

KEY DIFFERENCE: INCREMENTAL FORMULA

Why Incremental Works Mathematically

Traditional average:

$$Q(a) = \frac{\sum_{i=1}^n R_i}{n}$$

Incremental rewrite:

$$Q_n = Q_{n-1} + \frac{R_n - Q_{n-1}}{n}$$

Proof they're equivalent:

$$\begin{aligned}
 Q_n &= Q_{n-1} + (R_n - Q_{n-1}) / n \\
 &= (Q_{n-1} \times n) / n + (R_n - Q_{n-1}) / n \\
 &= (Q_{n-1} \times n + R_n - Q_{n-1}) / n \\
 &= (Q_{n-1} \times (n-1) + R_n) / n \\
 &= (\text{sum of first } n-1 \text{ rewards} + R_n) / n \\
 &= \text{Traditional average}
 \end{aligned}$$

INCREMENTAL STEP SIZE PROPERTIES

1. Natural Decay

N=1: step = 1.0 (100% change, fully adopt new info)
N=2: step = 0.5 (50% change)
N=5: step = 0.2 (20% change)
N=10: step = 0.1 (10% change)
N=100: step = 0.01 (1% change)

Automatic stabilization without parameter tuning!

2. Convergence Guarantee

Step size $\rightarrow 0$ as $N \rightarrow \infty$

This guarantees convergence to true mean:
With probability 1, $\lim Q_n = \text{true_mean}(a)$

3. Variance Reduction

Early (small N): Large steps, noisy but responsive
Late (large N): Small steps, smooth and stable

Example from Play 15:

Q = 7.5, error = -8.57
Step = 0.125 (only 12.5% adjustment)
Change = -1.07 (doesn't overreact to single bad pull)

COMPARISON: INCREMENTAL VS TRADITIONAL

Same final Q values, different paths

Both methods compute average of rewards:

Arm 2 rewards: [10, 10, 0, 10, 0, 10, 0, 10, 0, 10, 0, 10]
Average = 60/12 = 5.0

Final Q(2) converges to true mean (6.0) approximately

Advantages of incremental: 1. **Memory:** $O(1)$ vs $O(n)$ 2. **Speed:** $O(1)$ per update vs $O(n)$ 3. **Streaming:** Can update online without storing 4. **Numerical stability:** No summing large numbers

FORMULA REFERENCE

Incremental Update

$$Q_t = Q_{t-1} + \frac{R_t - Q_{t-1}}{N_t}$$

Alternative Form

$$Q_t = \frac{Q_{t-1} \times (N_t - 1) + R_t}{N_t}$$

Step Size Over Time

$$\text{Step size at play } t = \frac{1}{N_t}$$

Expected Error

$$E[|Q_n - \mu|] \approx \frac{\sigma}{\sqrt{n}}$$

(Standard deviation / sqrt(n) → errors decrease slowly)

SUMMARY: INCREMENTAL APPROACH

What Makes It Special

1. **Memory efficient:** O(arms) not O(arms × plays)
2. **Mathematically identical:** Same Q-values as storing all
3. **Natural decay:** Step size 1/N provides automatic stabilization
4. **Production ready:** Used in real systems

Step Size Intuition

- **Large N (well-learned):** Small step (don't change much)
- **Small N (new):** Large step (adapt quickly)
- **Automatic:** No parameter tuning needed

Performance (Same as -Greedy)

- Total reward: 120
- Arms found: 5
- Optimal arm: Yes
- Plays on optimal: 12

Key Insight

Incremental formulation is not a different algorithm—it's a more efficient way to compute the same Q-values. The ϵ -greedy behavior and final results are identical; only the computation method changed.

When to Use Incremental

- **Always** in production systems (memory matters)
- **Always** with streaming data (can't store history)
- **Always** with very long episodes (memory would overflow)
- **Standard choice** in modern implementations